

Step 1

This library database holds records for media, customers, events and employees.

- All books and similar assets fall under media, and are uniquely identifiable by an ID.
 - Potential media that could be added in the future is recorded in its own table
 - Authors are recorded in their own table, and a third table represents which books they participated in writing
- Customers each have an ID, and can loan a piece of media as long as their outstanding balance is zero. Their balance only increases if they do not return their loan on time.
- Employees are identified by their Email address, and their information is stored and kept in a table.
- Events are held by the library, and take place in social rooms. They have a target audience, which can be kids, teens, adults, seniors, or general. Customers can register for events.
- rooms can be booked by customers, as long as it does not interfere with an event being held in that room. Customers are limited to one booking per day.

Media

- Media (MediaID, Title, Type, PublicationDate)
- Authors (AuthorID, AuthorName)
- MediaAuthors (MediaID, AuthorID)
- CandidateMedia (CandidateID, Title, Type, PublicationDate)

Customers

- Customers (CustomerID, FirstName, LastName, DOB, Balance)

Loans

- Loans (MediaID, CheckoutDate, CustomerID, DueDate, ReturnDate)

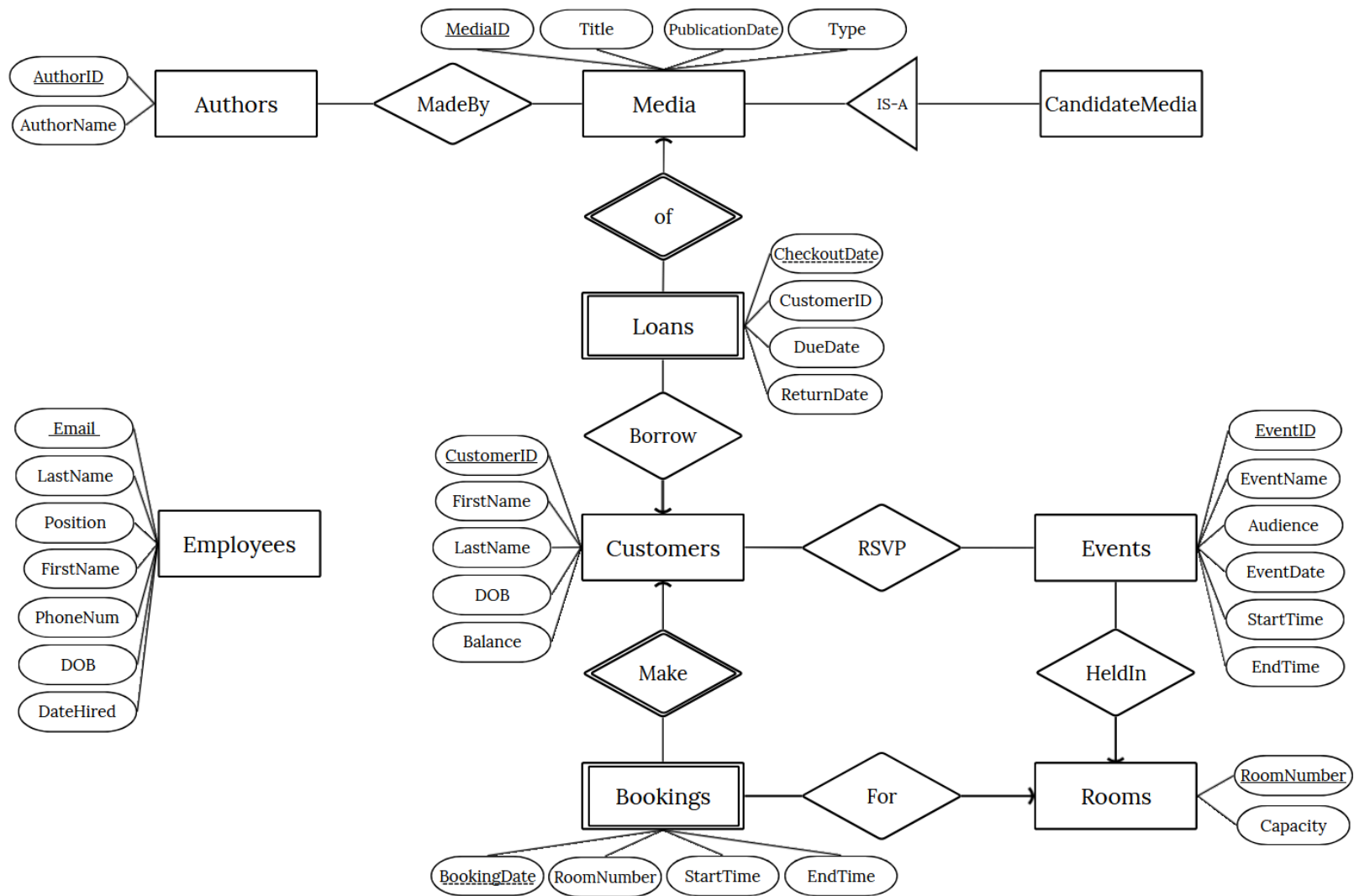
Rooms & Events

- Rooms (RoomNumber, Capacity)
- Bookings (CustomerID, BookingDate, RoomNumber, StartTime, EndTime)
- Events (EventID, EventName, Audience, RoomNumber, EventDate, StartTime, EndTime)
- EventRegistration(EventID, CustomerID)

Staff

- Employees (Email, FirstName, LastName, Position, PhoneNum, DOB, DateHired)

Step 2 (I have a more clear PNG)



Step 3

Confirming Each Table is in BCNF:

- Media
 - Primary Key: MediaID
 - All other attributes are functionally dependent on a superkey of the PK (in this case MediaID alone is enough), therefore this table is in BCNF.
- Authors
 - PK: AuthorID
 - The only other attribute is author name, which is FD on AuthorID.
- MediaAuthors
 - PK: {MediaID, AuthorID}
 - FDs: they're all trivial, so it's in BCNF.
- CandidateMedia
 - This table has the same attributes as Media, and is therefore in BCNF for the same reason.
- Customers
 - PK: CustomerID
 - FDs: CustomerID describes all other attributes, so this table is in BCNF.
- Loans
 - PK: {MediaID, CheckoutDate}
 - DueDate and ReturnDate functionally depend on the PK, and the CustomerID can be uniquely identified with the primary key as well. Therefore, since all the attributes depend on the PK, this table is in BCNF.
- Rooms
 - PK: RoomNumber
 - FD: there's only one other attribute and it depends on RoomNumber
- Bookings
 - PK: {CustomerID, BookingDate}
 - Since a customer can only book one room per date, CustomerID and BookingDate alone are enough to determine all other attributes. Thus, this table is in BCNF.
- Events
 - PK: EventID
 - FDs: All attributes of the event depend on the eventID, so this table is in BCNF
- EventRegistration
 - PK: {EventID, CustomerID}
 - FDs: they're all trivial, so this table is in BCNF.
- Employees:
 - PK: Email
 - FDs: All FD determinants are superkeys of Email, so this table is in BCNF.

Why Anomalies are not allowed:

Since every table is in BCNF, this prevents data redundancy. As a result:

- Insertion anomalies are prevented since each distinct entity is decoupled into an independent table, thus insertions are only dependent on functional dependencies. For example, a new room can be inserted into Rooms without requiring a pre-existing Booking record.
- Update anomalies are prevented since BCNF eliminates non-key redundancy. Each non-key attribute is only stored in one place under its respective candidate key. To update an attribute, you only need to update a single tuple, which prevents inconsistencies across the database.
- Deletion anomalies are prevented because independent entities are not bundled together in the same relation. For example, deleting a Loans record removes the borrowing transaction without accidentally deleting the Customer record or the Media entity from the database.